إعداد الطالبين:
1. **Med Adel Kouidi**
2. **Leila Zeghoune**

السنة الجامعية: 2026 / 2027

جامعة الشهيد حمة لخضر - الوادي | كلية العلوم الدقيقة | قسم الإعلام الآلي
مقياس: هندسة البرمجيات ونماذج التصميم (Design Patterns) - Master Informatique

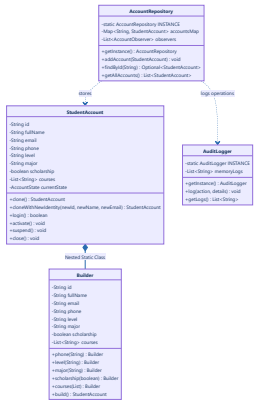
تقرير العمل التطبيقي: نماذج التصميم البرمجي (Design Patterns)

مشروع نظام إدارة الحسابات والخدمات الجامعية **(UniAccounts System)**

1. تقديم المشروع والأهداف العامة

يهدف مشروع **UniAccounts** إلى تصميم بنية برمجية متكاملة وقابلة للتوسيع لإدارة حسابات الطلبة، دورة حياة الحساب كاملة، اشتراكات الخدمات الطلابية، واستيراد بيانات الأنظمة القديمة مع التصدير بصيغ متعددة. تم تطبيق **10 نماذج تصميم معيارية (GoF Patterns)** موزعة على عملين تطبيقيين لحل مشكلات واقعية مع احترام مبادئ SOLID وتفادي الاقتران الوثيق (Loose Coupling).

2. الأنماط الإنشائية ومخطط الأصناف (TP 1 - Creational Patterns & UML)



الشكل 1: مخطط الأصناف (UML Class Diagram) للأنماط الإنشائية الثلاثة (Builder, Singleton, Prototype) وعلاقاتها بكامل تفاصيلها

1. نمط الباني (Builder Pattern) - الصنف StudentAccount.Builder

- **أين استعملته؟** كصنف داخلي ثابت متداخل `StudentAccount.Builder` مع واجهة بناء متسلسلة (Fluent API).
- **ما المشكلة التي حلها؟** كائن الحساب يحتوي على 8 حقول منها الإجباري والاختياري. المنشئات التقليدية المتتالية (Telescoping Constructors) تسبب خلطاً في ترتيب المعاملات، بينما يوفر الباني بناءً مرناً خطوة بخطوة مع تدقيق صحة البريد والمستوى ومنع تكرار المعرف.
- **ماذا كان سيحدث لو لم تستعمله؟** أخطاء برمجية في ترتيب المعاملات، صعوبة قراءة الكود، واضطرار المبرمج لتمرير معاملات فارغة (nulls).

2. نمط المفرد (Singleton Pattern) - الصنفان AuditLogger و AccountRepository

- **أين استعملته؟** في المستودع المركزي `AccountRepository` وسجل العمليات `AuditLogger` بأسلوب حامل الفئة Bill Pugh Holder.
- **ما المشكلة التي حلها؟** ضمان وجود مرجع وحيد ونقطة وصول عالمية لحفظ الحسابات ومنع تكرار المعرفات، مع توثيق التدقيق الأمني بأمان خيوط (Thread Safety) ودون إبطاء التزامن المفرط.
- **ماذا كان سيحدث لو لم تستعمله؟** تشتت البيانات بين نُسخ متعددة في الذاكرة، ضياع الحسابات، وفقدان التسلسل الأمني للتدقيق.

3. نمط النموذج الأولي (Prototype Pattern) - دالتا ()clone و ()cloneWithNewIdentity

- **أين استعملته؟** داخل الصنف `StudentAccount` لتطبيق استنساخ الحسابات مع النسخ العميق للمقررات (Deep Copy).
- **ما المشكلة التي حلها؟** تجهيز قالب حساب جاهز لكل دفعة وتخصص ومستوى، واستنساخه مباشرة للطلبة بتعديل المعرف والاسم فقط، دون إعادة إدخال قائمة المقررات المشتركة يدوياً.
- **ماذا كان سيحدث لو لم تستعمله؟** كلفة معالجة عالية وتكرار إدخال المقررات مئات المرات مع احتمالية وقوع أخطاء إدخال يدوية.

3. مخرجات تشغيل العمل الأول (TP 1) ومخطط حالات الحساب (State Machine)

لقطة شاشة مخرجات تشغيل برنامج MainWork1.java:

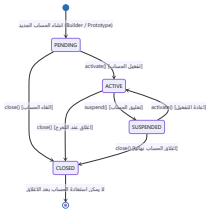
```
[1] Singleton Pattern:
repo1 == repo2     : true
logger1 == logger2 : true

[2] Builder Pattern:
Created: StudentAccount[id='20240001', name='Med Adel Kouidi', email='medadel.kouidi@univ-eloued.dz', level='L3',
major='Informatique (Genie Logiciel)', scholarship=true]
Testing validations:
Caught error (Email): Builder: email must contain '@'.
Caught error (Level): Builder: invalid level. Allowed: [M1, L1, M2, L2, L3]
Caught error (Duplicate ID): Duplicate Account ID: An account with ID '20240001' already exists!

[3] Prototype Pattern:
Template: StudentAccount[id='TEMPLATE_M1_IS', name='Template Batch IS', courses=[Data Warehousing, Advanced Databases, ERP
Systems, Information Security]]
Cloned 1: StudentAccount[id='20241001', name='Oualid Laroum', courses=[..., Cloud Computing]]
Cloned 2: StudentAccount[id='20241002', name='Farah Kouidi']
Template courses unchanged? true

--- Repository Accounts (3) ---
 - StudentAccount[id='20240001', name='Med Adel Kouidi']
 - StudentAccount[id='20241001', name='Oualid Laroum']
 - StudentAccount[id='20241002', name='Farah Kouidi']
```

4. دراسة أنماط العمل الثاني السلوكية (TP 2 - Behavioral Design Patterns)



الشكل 2: مخطط حالات الحساب (UML State Machine Diagram) يوضح دورة حياة الحساب وانتقالات الحالات بكامل تفاصيلها

4. نمط الحالة (State Pattern) - الواجهة AccountState والحالات الأربع

- **أين استعملته؟** في الواجهة `AccountState` والأصناف المنفذة: `PendingState`, `ActiveState`, `SuspendedState`, `ClosedState`.
- **ما المشكلة التي حلها؟** سلوك الحساب وقواعد تسجيل الدخول تتغير كلياً بحسب الحالة (منع الدخول للمعلّق أو المغلق). يوزع النمط السلوك على أصناف مستقلة تفوض الحساب تنفيذ العمليات بحسب حالته الراهنة، ويلغي شروط `switch/if-else` المتشعبة.
- **ماذا لو لم تستعمله؟** كود معقد مكرر وصعب الصيانة، مع خطورة ثغرات أمنية عند إضافة أي حالة جديدة مستقبلاً.

5. نمط المراقب (Observer Pattern) - الواجهة AccountObserver

- **أين استعملته؟** المراقبون: `AuditObserver` (سجل التدقيق)، `EmailNotifier` (البريد)، `AdminDashboard` (لوحة تحكم الإدارة).
- **ما المشكلة التي حلها؟** إشعار جهات متعددة ومستقلة تلقائياً عند إنشاء أو تفعيل أو تعليق الحساب وفق نموذج النشر/الاشتراك (Publish/Subscribe) دون إحداث اقتران وثيق بالمستودع.
- **ماذا لو لم تستعمله؟** تحول المستودع إلى كائن متضخم (God Object) واضطراره لمعرفة تفاصيل إرسال البريد وإحصاءات اللوحة مباشرة.

6. نمط الاستراتيجية (Strategy Pattern) - الواجهة ExportStrategy

- **أين استعملته؟** خوارزميتا التصدير: `CsvExport` و `JsonExport`.
- **ما المشكلة التي حلها؟** دعم تصدير بيانات الحسابات بصيغ مختلفة مع إمكانية تبديل الخوارزمية ديناميكياً أثناء وقت التشغيل (Runtime Interchangeability).
- **ماذا لو لم تستعمله؟** تداخل منطق التنسيق مع منطق حفظ البيانات وانتهاك مبدأ المسؤولية الواحدة ومبدأ الفتح/الإغلاق (OCP).

5. دراسة أنماط العمل الثاني البنائية (TP 2 - Structural Design Patterns)

7. نمط التركيب (Composite Pattern) - شجرة الهيكل التنظيمي للجامعة

- **أين استعملته؟** الواجهة الموحدة `UniversityComponent`، الأوراق `StudentLeaf`، والحاويات المركبة: `GroupComposite`, `DepartmentComposite`, `FacultyComposite`, `UniversityComposite`.
- **ما المشكلة التي حلها؟** تمثيل التسلسل الهرمي للجامعة ومعاملة الطالب المفرد والمجموعة المركبة بنمط موحد، واحتساب إجمالي الطلبة تلقائياً عبر دالة `countAccounts()` التراكمية.
- **ماذا لو لم تستعمله؟** كتابة دوال تكرار منفصلة ومعقدة لكل مستوى تنظيمي واستحالة حساب المجاميع التراكمية بمرونة.

8. نمط المحول (Adapter Pattern) - الصنف LegacyStudentAdapter

- **أين استعملته؟** لتحويل بيانات النظام القديم المجمد `LegacyStudentSystem` إلى واجهة `StudentImporter` النظامية.
- **ما المشكلة التي حلها؟** قراءة نصوص السجلات القديمة واستخراج الحقول وتحويلها إلى كائنات `StudentAccount` نظامية بالاستعانة بالـ Builder دون تعديل كود النظام القديم.
- **ماذا لو لم تستعمله؟** العجز التام عن استيراد بيانات الطلبة السابقين، أو تشويه معمارية النظام الجديد بحقول نصية غير متوافقة.

9. نمط المزخرف (Decorator Pattern) - الخدمات الطلابية الإضافية

- **أين استعملته؟** `ServiceDecorator` وتفريعاته: `LibraryDecorator` (500 دج)، `RestaurantDecorator` (1200 دج)، `InternetDecorator` (800 دج).
- **ما المشكلة التي حلها؟** تزويد الطلبة بتوليفات مختلفة من الخدمات الإضافية واحتساب الرسوم الشهرية والامتيازات تراكمياً في وقت التشغيل دون وراثة ثابتة.
- **ماذا لو لم تستعمله؟** حدوث انفجار في عدد الأصناف (Class Explosion) لإنشاء صنف لكل توليفة محتملة من الخدمات.

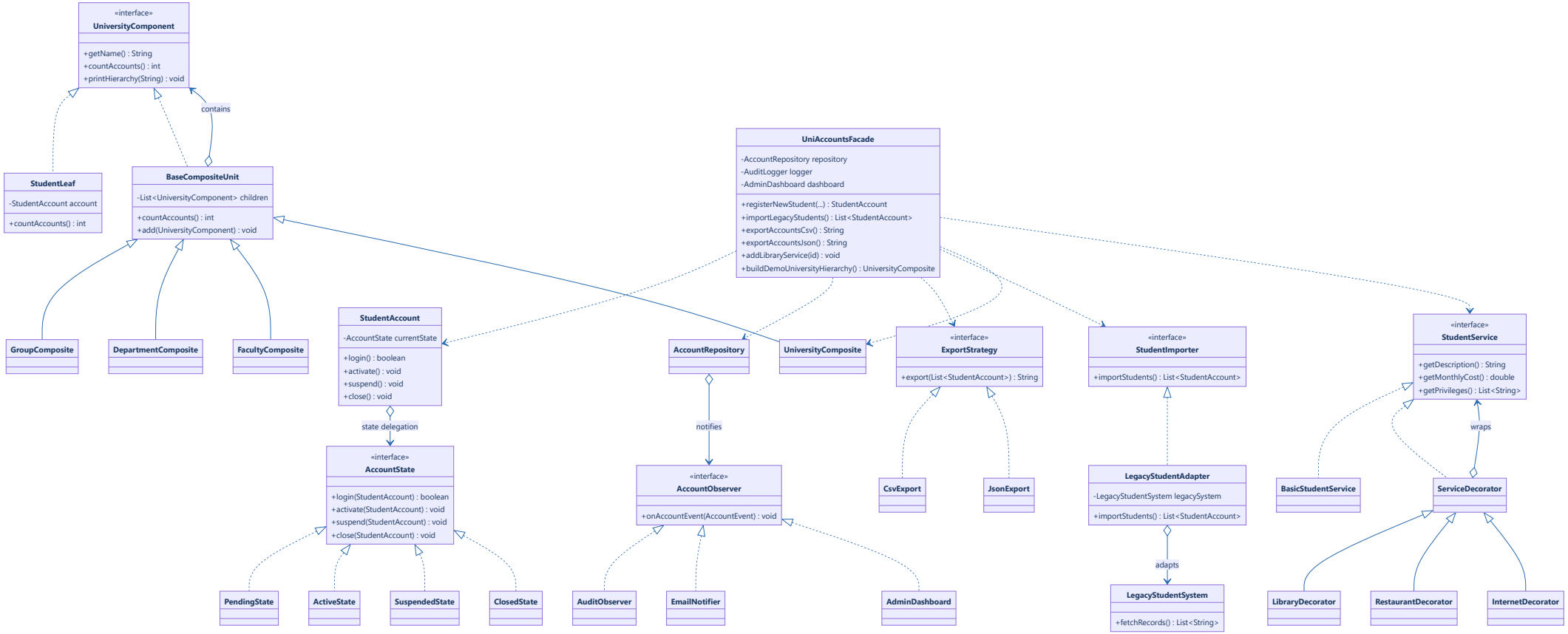
10. نمط الواجهة الموحدة (Facade Pattern) - الصنف UniAccountsFacade

- **أين استعملته؟** في الصنف `UniAccountsFacade` كنقطة مدخل موحدة لكافة عمليات النظام.
- **ما المشكلة التي حلها؟** إخفاء تعقيدات الأنماط التسعة السابقة وتوفير واجهة مستخدم مبسطة تتيح تنفيذ العمليات المعقدة بسطر واحد.
- **ماذا لو لم تستعمله؟** تعقيد كود العميل واضطرار المبرمج لمعرفة تفاصيل عشرات الأصناف والواجهات وتنسيقها يدوياً مع كل عملية.

جدول الربط المعياري الشامل بين الأنماط والأصناف في المشروع

النمط	نوعه	الأصناف والواجهات في الشفرة	الدور الوظيفي في نظام UniAccounts
Builder	إنشائي	`StudentAccount.Builder`	بناء الحساب والتحقق من صحة المدخلات
Singleton	إنشائي	`AccountRepository`, `AuditLogger`	مستودع وحيد مانع للتكرار وسجل تدقيق مركزي
Prototype	إنشائي	`StudentAccount.clone()`	استنساخ قوالب الدفعات السريعة بنسخ عميق
State	سلوكي	`AccountState`, `Pending`, `Active`, ...	إدارة دورة حياة الحساب وضوابط الدخول
Observer	سلوكي	`AccountObserver`, `Audit`, `Email`, `Dash`	إشعار تلقائي للبريد والتدقيق ولوحة التحكم
Strategy	سلوكي	`ExportStrategy`, `CsvExport`, `JsonExport`	تصدير الحسابات بصيغتي CSV و JSON
Composite	بنائي	`UniversityComponent`, `Group`, `Faculty`, ...	شجرة هيكل الجامعة واحتساب أعداد الطلبة آلياً
Adapter	بنائي	`LegacyStudentAdapter`, `LegacyStudentSystem`	استيراد بيانات النظام القديم غير المتوافق
Decorator	بنائي	`StudentService`, `Library`, `Restaurant`, ...	توليفات الخدمات الطلابية والرسوم الشهرية
Facade	بنائي	`UniAccountsFacade`	واجهة موحدة مبسطة تخفي تعقيدات النظام

6. مخطط الأصناف الشامل لكامل النظام (Comprehensive UML Class Diagram - 31 Classes & Interfaces)



الشكل 3: المخطط الشامل لكافة أصناف وواجهات النظام (جميع الأصناف الـ 31: State, Observer, Adapter, Composite, Strategy, Decorator, Facade) معروضة بالعرض الكامل (A4 Landscape) بدقة متجهة (Vector SVG) غير محدودة الوضوح.

7. لقطات شاشة مخرجات تشغيل العمل الثاني الشامل (MainWork2)

```
[1] State & Observer Demo:
[AUDIT LOG] [STATE_CHANGE] Account '20240010' (Leila Zeghoune) state changed from PENDING to ACTIVE.
[STATE] Account '20240010' is now ACTIVE.
  [EMAIL DISPATCH] To: Leila Zeghoune <leila.zeghoune@univ-eloued.dz> | Subject: Status now ACTIVATED
  [DASHBOARD UPDATE] New Event: ACTIVATED | Total ACTIVATED events: 1
-> Login while ACTIVE: [ACCESS GRANTED] Welcome Leila Zeghoune! Login successful.
-> Close account on graduation: [STATE] Account '20240010' has been CLOSED (Graduated).
-> Login while CLOSED: [ACCESS DENIED] Account '20240010' is CLOSED (Graduated). Login is forbidden.

[4] Decorator Pattern Demo (Student Services & Fees):
-------------------------------------------------
STUDENT SUBSCRIPTIONS: Nori Heni
Package     : Basic University Package for Nori Heni + [Library Service] | Monthly Fee: 500,00 DZD
Privileges  : Campus Ground Access, Student Card, Library Book Lending (Up to 5 books)
-------------------------------------------------
STUDENT SUBSCRIPTIONS: Mohammed Khadem
Package     : Basic Package + [Library Service] + [Restaurant Service] + [High-Speed Internet] | Monthly Fee: 2500,00 DZD
Privileges  : Library Lending, Daily Subsidized Meals, High-Speed Wi-Fi & Remote University VPN Access
-------------------------------------------------

[6] Composite Pattern Demo (Clean Hierarchy - 8 Target Students):
  + [University] Universite Hamma Lakhdar - El Oued (Count: 8)
     |+ [Faculty] Faculte des Sciences Exactes (Count: 8)
     |   |+ [Department] Departement d'Informatique (Count: 8)
     |   |   |+ [Group] Groupe 1 - Master Genie Logiciel (GL) (Count: 4)
     |   |   |   |- [Student] 20240010 : Leila Zeghoune [L3, CLOSED]
     |   |   |   |- [Student] 20240001 : Med Adel Kouidi [M2, ACTIVE]
     |   |   |   |- [Student] 20240002 : Oualid Laroum [M2, ACTIVE]
     |   |   |   |- [Student] 20240003 : Nori Heni [M2, ACTIVE]
     |   |   |+ [Group] Groupe 2 - Master Systemes d'Information (SI) (Count: 4)
     |   |   |   |- [Student] 20240004 : Mohammed Khadem [M2, ACTIVE]
     |   |   |   |- [Student] 20240005 : Farah Kouidi [M2, ACTIVE]
     |   |   |   |- [Student] 20240006 : Abdo Kacca [M2, ACTIVE]
     |   |   |   |- [Student] 20240007 : Abdo Louthri [M2, ACTIVE]

Total accounts in student tree: 8 | Faculte des Sciences Exactes : 8 accounts

[7] Admin Dashboard Statistics:
  CREATED : 8 | ACTIVATED : 9 | SUSPENDED : 1 | CLOSED : 1 | IMPORTED : 10 | TOTAL EVENTS : 29
```

8. توزيع المهام والعمل المنجز بين الطالبين (Work Distribution)

عضو الفريق	المهام والأنماط البرمجية المنجزة
الطالب: Med Adel Kouidi	▪ تصميم الصنف الأساسي `StudentAccount` ونمط `Builder` مع دوال الفحص والتحقق الأمني للمدخلات. ▪ تطبيق نمط `Singleton` للمستودع `AccountRepository` ومسجل التدقيق `AuditLogger` (Bill Pugh Holder). ▪ هندسة دورة حياة الحساب وقواعد الأمان وتطبيق نمط `State` (Pending, Active, Suspended, Closed). ▪ تطوير نمط المحول `Adapter` لاستيراد سجلات النظام الجامعي القديم `LegacyStudentSystem`. ▪ بناء واجهة البوابة الموحدة `UniAccountsFacade` وبرنامج اختبار العمل الأول `MainWork1`.
الطالبة: Leila Zeghoune	▪ تطبيق نمط النموذج الأولي `Prototype` واستنساخ قوالب الدفعات مع النسخ العميق للمقررات (Deep Copy). ▪ بناء نمط المراقب `Observer` بمشتركيه الثلاثة: التدقيق `Audit`، البريد `Email`، ولوحة الإدارة `Dashboard`. ▪ تطوير استراتيجيات التصدير بنمط `Strategy` لصيغتي `CsvExport` و `JsonExport`. ▪ بناء نظام الاشتراكات والخدمات الطلابية وتوليفات الرسوم الشهرية بنمط `Decorator`. ▪ بناء الشجرة الهرمية للجامعة بنمط `Composite` وتصميم برنامج اختبار العمل الثاني الشامل `MainWork2`.